Universidad Metropolitana

Departamento de Energía y Automatización

Robótica Industrial

PRÁCTICA Nº 4

PiCar-X con EzBlock Studio

Asignatura: Robótica Industrial (FPTEN27)

Profesor: José Montilva

Laboratorio de Robótica Industrial

Caracas, noviembre de 2026

Guía de Práctica de Laboratorio

Práctica #4 — PiCar-X con EzBlock Studio (programación por bloques)

Objetivo General

Programar el robot móvil PiCar-X mediante bloques en EzBlock Studio, pasando de ejecutar los ejemplos del fabricante a construir comportamientos propios que integren movimiento, sensores y visión, e identificar la instrucción de Python que genera cada bloque.

Antes de empezar — El PiCar-X y EzBlock Studio

El PiCar-X es un robot móvil de SunFounder controlado por una **Raspberry Pi** con una placa de expansión **Robot HAT**. Tiene dos **motores DC** traseros para la tracción, un **servomotor** delantero para la dirección y dos servomotores más que orientan la cámara (pan horizontal y tilt vertical). Como sensores incorpora un **ultrasónico** al frente, un módulo de **escala de grises** de 3 canales debajo del chasis y una **cámara**.

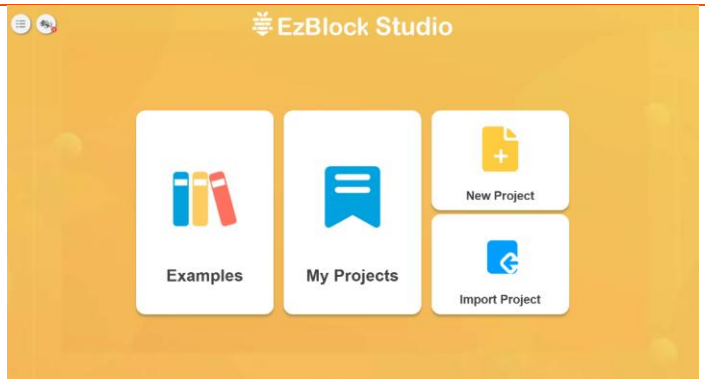
EzBlock Studio es un entorno de programación visual basado en Blockly (la misma base de Scratch). Cada bloque que arrastras se traduce en código Python que se ejecuta en la Raspberry Pi del carro.

⚠ Memoria SD correcta

- Esta práctica usa la memoria SD de **EzBlock**, que es distinta a la de Python. Verifica que sea la correcta **antes** de encender el carro.

Conexión

1. Inserta la memoria SD de EzBlock en la Raspberry Pi y enciende el carro con el interruptor del Robot HAT. Espera el sonido de encendido.
2. Obtén la dirección IP del carro: conecta monitor, teclado y mouse a la Raspberry Pi e inicia sesión con usuario `raspberry` y clave `pi`.
3. En tu laptop abre el navegador y entra a **http://ezblock.cc/ezblock-studio**.
4. Selecciona el ícono del carrito para conectarte con el PiCar-X. Una vez conectado, entra a **Examples** para cargar los ejemplos.



Pantalla principal de EzBlock Studio

Cómo trabajar cada ejercicio

- Carga el ejemplo, pulsa **Run** y observa qué hace el carro **antes** de abrir los bloques.
- Abre el editor y ubica qué bloque produce cada acción. Activa la vista de **código Python** para ver la instrucción equivalente.
- Los retos se resuelven en un **proyecto nuevo** del grupo. No sobrescribas los ejemplos originales: el siguiente grupo los necesita.

⚠ Seguridad

- Trabaja en el piso o con espacio libre alrededor del carro; nunca lo pongas a andar sobre una mesa.
- Ten siempre a mano el botón **Stop** de EzBlock o el interruptor del Robot HAT para detenerlo.

Ejercicio #1 Movimiento en lazo abierto

Los motores DC del PiCar-X reciben una señal PWM: a mayor ciclo de trabajo, mayor velocidad. Pero el carro **no tiene encoders**, así que no sabe cuánto avanzó realmente. Mover el carro "por tiempo" es un control en **lazo abierto**.

Paso 1 — Ejemplo Move

Carga **Examples** → **Move** y ejecútalo. El carro avanza, retrocede, gira a la izquierda, gira a la derecha y se detiene. Abre los bloques y relaciona cada uno con su función de Python (por ejemplo, el bloque de avance genera `px.forward(velocidad)`).

Paso 2 — Ejemplo Remote Control

Carga **Examples** → **Remote Control** y conduce el carro con los controles de la pestaña de control remoto. Observa cómo los valores del joystick se convierten en velocidad y ángulo de dirección.

Paso 3 — Modifica Move

Cambia la velocidad (prueba 10, 50 y 80) y el ángulo de dirección. El servo de dirección acepta aproximadamente de −30° a +30°: ¿qué pasa si pides un valor mayor?

★ Reto — Cuadrado de 50 cm

Marca con cinta un punto de partida en el piso. Programa al carro para recorrer un cuadrado de unos 50 cm de lado usando un bloque de **repetición** (4 veces: tramo recto + giro de 90°).

- Ajusta los tiempos de avance y de giro hasta que el cuadrado se parezca lo más posible.
- Ejecuta el programa **3 veces sin cambiar nada** y mide cuánto queda el carro del punto de partida.

Intento	Velocidad	t recto (s)	t giro (s)	Distancia al punto de partida (cm)
1				
2				
3				

Registra estos valores en la bitácora del grupo.

⚠ ¿Qué observar?

- ¿Por qué el carro no vuelve exactamente al mismo punto si el programa es idéntico?
- Piensa en el deslizamiento de las ruedas, el estado de la batería y la superficie. ¿Qué sensor le haría falta al carro para corregir ese error?

Ejercicio #2 Sensores: ultrasonido y escala de grises

El sensor **ultrasónico** emite un pulso de sonido y mide el tiempo que tarda el eco en volver; la distancia es d = v·t / 2 (v ≈ 343 m/s). El módulo de **escala de grises** tiene 3 pares emisor-receptor infrarrojo que miden cuánta luz refleja la superficie: una superficie clara refleja mucho y una oscura poco.

Paso 1 — Test Ultrasonic Module

Carga el ejemplo y coloca una caja o un cuaderno a distancias conocidas, medidas con regla.

Distancia real (cm)	Lectura del sensor (cm)	Diferencia (cm)
10		
20		
40		
80		

Registra estos valores en la bitácora del grupo.

Paso 2 — Test Grayscale Module

Carga el ejemplo y anota los tres valores (izquierdo, central, derecho) sobre cada superficie.

Superficie	Izquierdo	Central	Derecho
Piso / papel blanco			
Cinta aislante negra			
Hoja de papel negro			

Registra estos valores en la bitácora del grupo.

Paso 3 — Cliff Detection

Este ejemplo usa el mismo sensor de escala de grises para detectar un "precipicio" (cuando casi no vuelve luz). Ejecútalo en el piso y acerca el carro a la hoja de papel negro. **No lo pruebes en el borde de una mesa.**

★ Reto — Estacionamiento con aviso

Programa al carro para que avance hacia una pared y:

- avance mientras la distancia sea **mayor a 20 cm**;
- se detenga y diga **"Stop"** (bloque de texto a voz) al entrar en la zona de 10 a 20 cm;
- retroceda si la distancia es **menor a 10 cm**.

Pista: necesitas un bloque **if / else if / else** dentro de un ciclo **forever**.

⚠ ¿Qué observar?

- ¿Por qué el carro confunde la hoja negra con un precipicio? ¿Qué mide realmente el sensor?
- ¿Qué ocurre con el ultrasonido frente a una superficie blanda (un suéter) o inclinada? ¿Por qué?

Ejercicio #3 Seguimiento de línea

Para seguir una línea, el carro compara los 3 sensores de escala de grises: si la línea está bajo el sensor central avanza recto; si se corre hacia un lado, gira hacia ese lado para recuperarla. Es un comportamiento **reactivo**: leer sensores → decidir → actuar, una y otra vez.

Con cinta aislante negra arma un circuito cerrado sobre una superficie clara, con al menos una curva abierta y una curva cerrada.

Paso 1 — Minecart

Carga **Examples** → **Minecart** y colócalo sobre el circuito. Observa en qué tramos funciona y en cuáles se sale.

Paso 2 — Minecart Plus

Carga **Examples** → **Minecart Plus** en el mismo circuito. Esta versión agrega lógica de recuperación cuando el carro pierde la línea. Compara ambos programas: ¿qué bloques adicionales tiene la versión Plus?

Paso 3 — Velocidad vs. precisión

En Minecart Plus cambia la velocidad y registra el resultado.

Velocidad	¿Completa la vuelta?	Tiempo de vuelta (s)	Veces que pierde la línea
10			
30			
50			

Registra estos valores en la bitácora del grupo.

★ Reto — Línea con frenado de emergencia

Partiendo de Minecart Plus (en un proyecto nuevo), agrega el sensor ultrasónico:

- si hay un obstáculo a **menos de 20 cm**, el carro se detiene aunque esté sobre la línea;
- cuando retiras el obstáculo, retoma el seguimiento de línea por sí solo.

⚠ ¿Qué observar?

- ¿Por qué al aumentar la velocidad el carro pierde la línea en las curvas cerradas?
- Si cambia la iluminación del salón o el color del piso, ¿qué parte del programa habría que ajustar?

Ejercicio #4 Visión: colores y rostros

La cámara del PiCar-X procesa cada imagen en la Raspberry Pi (con funciones de OpenCV) y entrega la **posición** y el **tamaño** del objeto detectado. Con esa información el carro puede tomar decisiones. Abre el **Video Monitor** de EzBlock para ver lo que ve la cámara.

Paso 1 — Color Detection

Carga el ejemplo, elige un color objetivo y prueba con objetos del laboratorio. Solo se detecta un color a la vez.

Color	¿Lo detecta?	Distancia máxima de detección (cm)
Rojo		
Azul		
Verde		
Amarillo		

Registra estos valores en la bitácora del grupo.

Paso 2 — Face Detection y Say Hello

Ejecuta **Face Detection** y luego **Say Hello**: cuando el carro detecta un rostro, asiente con la cámara y dice "Hello". Prueba a distintas distancias, de perfil y cubriendo parte de la cara.

Paso 3 — Bullfight

En **Bullfight** el carro persigue un objeto rojo (la "capa"): usa la posición horizontal del color para orientar la cámara y la dirección. Mueve el objeto lentamente de un lado a otro y observa cómo responde.

★ Reto — Cruce peatonal

Programa al carro para que avance en línea recta y se comporte como un vehículo frente a un paso peatonal:

- si detecta un rostro (tu cara o una foto impresa), se detiene y dice **"Please cross"**;
- espera a que no haya ningún rostro durante unos 2 segundos y reanuda la marcha.

Pista: una sola detección puede ser un falso positivo. Usa una **variable contador** y detente solo si detectas el rostro varias veces seguidas (por ejemplo, más de 3 de 10 lecturas).

> ⚠ **¿Qué observar?**
> - ¿Por qué conviene confirmar la detección varias veces antes de frenar?
> - ¿Qué color se detectó con más facilidad y cómo influye la iluminación?

Al finalizar

1. Detén el programa con **Stop** y espera unos segundos antes de apagar el carro con el interruptor del Robot HAT.
2. Retira la memoria SD de EzBlock y guárdala en su estuche.
3. Deja el circuito de cinta despegado y el puesto ordenado.